What the firmware image says about the DMM6500's Lua API

`docs/UNDOCUMENTED.md` holds behaviour measured on the instrument, and every claim in it has been to the bench. This file is the other half: what can be read out of the **firmware image** — names, constants, values and structure that no Keithley document mentions. The two have different evidence bars on purpose, and mixing them would wreck the older file's.

The image is the only complete census available. On the instrument `next(display)` returns zero keys: the namespace is metatable-backed, so a name that has not been guessed cannot be discovered from a running DMM6500 at all.

Status	Meaning
`IMAGE`	Read out of a firmware table, where the name and its number are in the same record.
`CODE`	Read from the firmware's instructions or decompiled C.
`CONFIRMED`	`IMAGE` or `CODE`, and then checked on this DMM6500 at 1.7.17a.
`ABSENT`	In the image, and **not** present on this model — the pool is shared across Keithley's line.
`HAZARD`	Known to crash, wedge or saturate something. Stated with what to do instead.

Nothing in the calibration or adjustment namespaces is listed, probed or described here. Exclusion is token-wise, so `display.scalex`, `display.scaley` and `localnode` stay in — they merely contain the letters.

The machine the API is talking to

Two identical 400 MHz PowerPC processors sit on the CPU board with an FPGA that drives the display and touchscreen. The processor with 128 MB of DDR2 is the raster graphics engine; the other, with 256 MB, runs the application and the UI. The analog front end has its own ARM Cortex-M4.

The firmware bundle names all of it. `versions.txt` inside `ki_DMM6500_v1_7_18a.zip`:

Component	Version	What it is
`6518`	v1.7.18a	the main board — `main.zx`, the application CPU
`afpgaalgo_1.sea`, `afpgadata_2.sed`	v0.36.0a	the analog FPGA
`amicro_3.zx`	v0.10.0a	the analog micro, the Cortex-M4
`6520` / `6521` / `6522`	v0.12.0a / v0.9.0a / v0.6.0a	the GPIB, TSP-Link and RS-232 comm cards, each with its own FPGA pair

This matters for every drawing call. A display call on the application CPU is a message to a different processor, so the cost is per *call*, not per pixel. That is the reason `display.setpixel` is too slow to draw with while the graph widget is fast: one graph call hands over a whole trace. The graphics engine can also be **saturated and crashed by being fed too much too quickly**, which is why this project holds its own display updates between 1 and 4 Hz.

Getting at the image

Nothing is encrypted. `file(1)` calls the decoded result an "OpenPGP Secret Key", which is a magic-byte coincidence.

```
ki_DMM6500_v1_7_18a.zip
  └ main.zx            itself a zip
      └ main.x         Motorola S-record, hex ASCII  (27.5 MB)
          └ decode     10,710,276 bytes in 11 runs, 0 bad lines -- the application CPU
  └ logic.zx           a zip holding logic.x, also S-records -- a Lattice FPGA bitstream
  └ rom.zip            holds afpgaalgo_1.sea and friends -- not yet examined
```

logic.zx is a Lattice bitstream, not Xilinx

`logic.x` decodes to exactly 1 MB at flash `0xFE000000` — one contiguous run, no gaps — of which `0xFE000000`-`0xFE091000` is populated and the rest is erased. It holds **no CPU instructions at all**: zero `mflr`, zero `blr`, zero ARM or MIPS return idioms, and 44 % of it is `0xFFFFFFFF`.

The layout is a 4-byte big-endian payload length, `0xFF` padding, then the bitstream:

```
FE000000  00 09 08 4F                     591,951 -- the exact payload length
FE000004  ff ... ff                       341 bytes of padding
FE000159  bd b3 ff ff ff ff 3b 00 00 00   0xBDB3 is the Lattice preamble
FE000167  e2 00 00 00 41 11 10 43         0xE2 verify-id, then IDCODE 0x41111043
```

`IMAGE`. **`0xBDB3` is Lattice Semiconductor's bitstream preamble.** Xilinx would begin `0xAA995566`, which does not occur anywhere in the file in any bit or byte order — I checked the plain image, the byte-swapped form, and the whole image bit-reversed, since a configuration PROM image is commonly stored bit-reversed. The IDCODE's low 12 bits, `0x043`, are Lattice's JTAG manufacturer field; the part field is `0x1111` and the version 4. There is exactly one preamble, so one device. A 65-byte autocorrelation peak in the payload fits frame-based configuration data.

The file carries no ASCII part name, design name or date — a Lattice `.bit` normally opens with a comment block naming the device, and this one has been stripped to padding.

`tools/` has nothing for this; the scripts live outside the repo because an extracted firmware archive is untrusted data. `/tmp/ghsc/srflat.py` flattens the S-records at their true addresses, which `desrec.py` did not do — it concatenated them, so every offset past the first gap was wrong.

The image runs from DDR at zero, not from flash

The S-record addresses are flash: eleven runs from `0xFF200000` to `0xFFF75EAC`, with the PowerPC reset vector at `0xFFF00100`. **The code does not run there.** 37,512 of the 39,898 addresses its `addis`/`addi` pairs build are in `0x00xxxxxx`: the image is copied into DDR and linked low.

The mapping was recovered by alignment rather than assumption. For a candidate delta, score how often a materialised address lands exactly on the first byte of a NUL-terminated string:

delta	lands on a string start
`0xFF200000`	**13.8 %**
`0xFF20A258`	2.8 %
`0xFF1FE96C`	2.8 %

delta	lands on a string start
four others	2.8 - 3.2 %

So `link = flash − 0xFF200000`, the image is linked at `0x00000000`, and an importer told the flash base resolves nothing. The four large runs are contiguous in link space, so `main.x` is the **application** CPU alone; the raster engine's code is not in it.

The image states this itself, which beats inferring it. It carries its own section table — a Green Hills `.secinfo` at flash `0xFFBE2300` — giving `.text` at RAM `0x4000`, `0x5217D0` long, with the flash copy exactly `0xFF200000` higher. Two independent routes to the same delta, one of them the linker's own record. `IMAGE`. The scoring table above is kept because it is the method that works on an image with no section table at all.

Loading it into Ghidra, after `srflat.py` has written the app region:

```
analyzeHeadless <proj> <name> -import main_app.bin \
    -processor PowerPC:BE:32:default -loader BinaryLoader -loader-baseAddr 0x0
```

Ghidra 12.2 ships no macOS decompiler native. It builds from the bundled source in about a minute: `make -j10 ghidra_opt` in `Ghidra/Features/Decompiler/src/decompile/cpp`, then copy `ghidra_opt` to `Ghidra/Features/Decompiler/os/mac_x86_64/decompile`. No clone and no gradle needed.

How the API is registered, and why the pool over-reports it

Constants come from static tables of 8-byte records, `{ const char *name; int32_t value; }`, NULL-terminated and grouped by family. The names are stored fully qualified — `display.KEY_HELP`, `dmm.SLOPE_RISING` — so a table can be found by its shape alone, and a row cannot pair a name with somebody else's number. 742 of them are recovered this way across 23 namespaces: `dmm` 293, `display` 130, `buffer` 84, `trigger` 65, `channel` 31, `scan`, `ki`, `serial`, `analogtrigger`, `format`, `localnode`, `file`, `filter`, `math`, `tspnet`, `coupling`, `lan`, `rel`, `fan`, `threshold` and three singletons.

Functions are not a table. Each one is built at startup by a sequence of method calls on a descriptor object — set the name, append several strings, set a double — inside one large registrar per namespace. `display`'s is at link `0x002CC0B4`; the three-instruction stub at `0x002CC0A8` in front of it is a tail call, and an auto-analyser that believes the first call never returns will truncate the function there and show you two lines of C. No handler address appears in those blocks, so what is being built is command **metadata** and dispatch happens later by name.

The string pool is shared across Keithley's product line, so its names are not this model's API. Measured: `dump`, `capture`, `scalex`, `getpixel`, `drawstyle`, `generate`, `getobject`, `removeall`, `select`, `cursor`, `add`, `markersize`, `scaletrace`, `getchecksum`, `autolocal` and `press` are all in the `display` pool and all `nil` on a DMM6500 at 1.7.17a. A name from the image is a candidate; a `type()` lookup on the instrument is the answer. That lookup is also the safe way to ask, since it is a table read and never a call.

A constant is userdata wrapping its number, and `tonumber` gets the number out. Measured:

```
type(display.OBJ_CIRCLE)      userdata
tostring(display.OBJ_CIRCLE)  "display.OBJ_CIRCLE"
tonumber(display.OBJ_CIRCLE)  4
display.OBJ_CIRCLE == 4       false
```

So the integers in this file are both the firmware's internal ids and the values Lua can obtain. The comparison is false because Lua 5.0 only consults `__eq` when both operands have the same type — compare with `tonumber(x) == 4`, never `x == 4`.

This interpreter is a modified Lua 5.0.2. The metatable on these objects carries `__tostring`, `__tonumber`, `__eq`, `__add`, `__sub`, `__mul`, `__div`, `__unm` and `__concat`. Stock 5.0.2 has no `__tonumber` metamethod at all, so Keithley patched the core; and the arithmetic metamethods mean a constant can be added and concatenated directly, which is worth knowing before writing `tonumber()` everywhere.

A dotted string in the image is not an API name. The 10.7 MB holds about a thousand of them, and the residue includes source filenames (`interruptcontroller.c`, `serial_iodevice.c`), lower-case attribute paths used only when the instrument generates a configuration script (`filter.enable`, `rel.level`), and substrings sitting inside help text. `buffer.FILL_ONCE` appears at an unaligned address with no reference anywhere in the image — it is a literal the instrument prints into a generated script, while the real constant is registered separately under its short name. So a dotted-name count is a starting point for a census and never the census itself.

The full catalog is `docs/firmware-constants.tsv`, with a `kind` column, because two different things share these tables. A final component in capitals is a constant — `display.KEY_HELP`. A lower-case one is an **attribute path** — `analogtrigger.edge.level` — whose number is a default or an offset rather than anything Lua reads. And 73 names appear in more than one table with different numbers, which means at least one of those tables is not a constant registry: every row of such a name is marked `DUP` so that nothing quotes one as a fact. **No `display.*` or `ki.*` constant is among them**, so everything in this file comes from a single table.

---

display: what exists and is undocumented

`CONFIRMED` by `type()` on 1.7.17a. None of these appears in either reference manual, the user manual, `DMM6500DisplayDoc.md` or `Display API Command Reference.md`.

Name	Kind	Note
`display.setfill`	function	takes a `FILL_*` direction
`display.setcell`	function	
`display.setfont`	function	the fourteen font constants; see `UNDOCUMENTED.md` for their measured heights
`display.setpixel`	function	`HAZARD`: measured too slow to draw with. Use `display.graph`
`display.setposition`	function	
`display.setthickness`	function	
`display.setminmax`	function	
`display.setabbrv`	function	

Name	Kind	Note
`display.setpopupmenu`	function	
`display.setevent`	function	takes an OR of the `EVENT_*` flags below
`display.setkeyenable`	function	measured in `UNDOCUMENTED.md`: disables a key, no getter
`display.flush`	function	not yet characterised — the one candidate for batching draws
`display.graph`	table	a sub-namespace, below
`display.activebuffer`	table	
`display.readingformat`	userdata	a property; reads back `display.FORMAT_PREFIX`
`display.lightstate`	userdata	a property; reads back `display.STATE_LCD_25` — the backlight
`display.watchchannels`	string	a property; empty on this instrument

display.graph

`CONFIRMED`. The graph widget is the fast drawing path: one call hands a whole trace to the graphics processor instead of a message per pixel.

Confirmed function	Confirmed `nil`
`graph.scalex`, `graph.scaley`, `graph.scaletrace`	`graph.setminmax`, `graph.generate`
`graph.drawstyle`, `graph.markersize`, `graph.markerlastcolor`	`graph.getobject`, `graph.xscale`, `graph.yscale`
`graph.select`, `graph.cursor`, `graph.add`, `graph.removeall`	`graph.cursorlock`

`HAZARD`: **setting the graph's horizontal units to frequency crashes the instrument.** Time in microseconds works and is what to use. Measured on this bench before this document existed.

The scaling constants live on `display`, not on `display.graph`: `XSCALE_LATEST`, `XSCALE_GROUP`, `XSCALE_ALL`, `XSCALE_AUTOBIN`, `XSCALE_SWIM`, `XSCALE_OFF`, and `YSCALE_SMART`, `YSCALE_ALL`, `YSCALE_PER_TRACE`, `YSCALE_PER_TRACE_ONCE`, `YSCALE_SWIM`, `YSCALE_OFF`.

Object types, in full

`IMAGE`. The documented set is `OBJ_TEXT`, `OBJ_RECT`, `OBJ_LINE`, `OBJ_BUTTON`, `OBJ_BUTTON_MENU`, `OBJ_SCREEN`, `OBJ_SCREEN_MENU`, `OBJ_SWIPE` and the `OBJ_EDIT_*` family. The rest are not.

id	name	id	name
1	`OBJ_TEXT`	20	`OBJ_SCREEN`
2	`OBJ_RECT`	21	`OBJ_SCREEN_MENU`
3	`OBJ_LINE`	25	`OBJ_SWIPE`
4	**`OBJ_CIRCLE`**	**26**	**`OBJ_TIMER`**
5	**`OBJ_IMAGE`**	**30**	**`OBJ_PROGRESS_BAR`**
6	`OBJ_BUTTON`	**31**	**`OBJ_POPUP_MENU_SCREEN`**
7	`OBJ_BUTTON_MENU`	32	`OBJ_EDIT_NOTIFY_EVENT`

id	name	id	name
8	`OBJ_EDIT_NUMBER`	33	`OBJ_EDIT_WAIT_EVENT`
9	`OBJ_EDIT_OPTION`	34	`OBJ_EDIT_BUFFER`
10	`OBJ_EDIT_STRING`	35	`OBJ_EDIT_CHANNEL`
11	`OBJ_EDIT_CHECK`	**36**	**`OBJ_LIST`**
12	**`OBJ_EDIT_SLIDER`**		

`OBJ_CIRCLE`, `OBJ_IMAGE` and `OBJ_PROGRESS_BAR` are the interesting ones: a circle and an image are drawing primitives expressed as objects, which the graphics processor draws in one message, and a progress bar is a widget this project currently fakes with a rectangle.

Event flags are a bitmask

`IMAGE`, and the values say so — they are powers of two, so `display.setevent` takes an OR.

flag	value	flag	value
`EVENT_PRESS`	1	`EVENT_ENDAPP`	16
`EVENT_SCALE`	2	`EVENT_SHOW`	32
`EVENT_DRAG`	4	`EVENT_HIDE`	64
`EVENT_CURSOR`	8		

`EVENT_SHOW` and `EVENT_HIDE` are not mentioned anywhere outside the image.

Other display families worth knowing

`IMAGE`. Full list with values in `docs/firmware-constants.tsv`.

- `FONT_SMALL` 0, `FONT_MEDIUM` 1, `FONT_LARGE` 2, `FONT_HUGE` 3, `FONT_1` - `FONT_10` 4-13. Fourteen faces, and `UNDOCUMENTED.md` has their measured pixel heights — the ten numbered ones are not a size ladder.
- `JUST_LEFT` 0, `JUST_RIGHT` 1, `JUST_CENTER` 2 — text justification.
- `FILL_LEFT` 0, `FILL_RIGHT` 1, `FILL_UP` 2, `FILL_DOWN`.
- `STYLE_LINE` 0, `STYLE_MARKER` 1, `STYLE_BOTH` 2 — for `graph.drawstyle`.
- `STATE_ENABLE` 0, `STATE_DISABLE` 1, **`STATE_READONLY` 2**, `STATE_INVISIBLE` 3.
- `MARK_GREEN` 0, `MARK_YELLOW` 1, `MARK_RED` 2; `CURSOR_VERTICAL`, `CURSOR_HORIZONTAL`, `CURSOR_TRIGGER`.
- `BUTTON_OPTION1` - `BUTTON_OPTION10` 5-14; `BUTTONS_NONE` 0 through `BUTTONS_YESNOCANCEL` 5.
- `SFORMAT_ANY` 0 … `SFORMAT_FILE_NAME` 4, **`SFORMAT_IP_ADDRESS` 5**. This project shipped a two-field address editor because 21 name guesses missed `SFORMAT_IP_ADDRESS`; the image had it.
- `TYPE_NONE` 0, `TYPE_SCREEN` 1, `TYPE_BOTH` 3.
- `DUMP_NONE` 0, `DUMP_LIST` 1, `DUMP_BRIEF` 2, `DUMP_ALL` 3 — `ABSENT`, along with `display.dump`.

---

ki: the internal namespace, and the front-panel key table

`ki` is a table and `ki.display` is a table — `CONFIRMED`. It holds constants and nothing else: 29 names, every one of them an action code for a physical input. None is documented anywhere.

The expansion of `ki` is not in the image. "KeithleyInternal" appears nowhere in 10.7 MB; the only strings are `Keithley` and `keithley`. So the reading is plausible and unconfirmed.

Input	press	release
HOME	1	11
MENU	2	12
QUICKSET	3	13
HELP	**4**	**14**
ENTER	5	15
EXIT	6	16
FUNCTION	7	17
TRIGGER	8	18
FRONTREAR	9	19
OUTPUT	10	20
TOUCH	25	26
2TOUCH	33	34

Also `ACTION_MOVE_TOUCH` 32, `ACTION_MOVE_2TOUCH` 35 and `ACTION_VELOCITY` 36 for drag and swipe, and `ACTION_KNOB_LEFT` 21 with `ACTION_KNOB_RIGHT` 22 — **for a rotary knob the DMM6500 does not have.** The knob is on the DMM7510 and the SMUs. That is the clearest single piece of evidence that these tables are one shared codebase across Keithley's line, and the reason a name in the image is only ever a candidate until the instrument is asked.

The action codes and `display.KEY_*` are the same table. `display.KEY_HELP` is 4, which is `ki.display.ACTION_PRESS_HELP`; `KEY_TRIGGER` is 8, `KEY_FUNCTION` 7, `KEY_QUICKSET_APPS` 3. The documented `KEY_*` set names four of the twelve inputs, and `8` is also the `button=8` the instrument's own web panel posts to press TRIGGER, so the web interface drives this same table.

Using the HELP key: it cannot be done, and here is the evidence

No front-panel key except TRIGGER can reach a running Lua program on this firmware. That is a negative result, and it is worth as much as a feature, because the alternative is somebody spending a session looking for the route.

Four independent things close it off.

route	state
a trigger event from the key	`trigger.EVENT_DISPLAY` is the **only** display-sourced trigger event in the image. The complete list is `ANALOGTRIGGER`, `BLENDER`, `COMMAND`, `DIGIO`, `DISPLAY`, `EXTERNAL`, `LAN`, `NONE`, `NOTIFY`, `SCAN_ALARM_LIMIT`, `SCAN_CHANNEL_READY`, `SCAN_COMPLETE`, `SCAN_MEASURE_COMPLETE`, `TIMER`, `TSPLINK`. There is no per-key event and no `EVENT_KEY`.

route	state
`display.setkeyenable`	takes a key **away** from the firmware. It cannot report a press, and there is no `display.getkeyenable` — measured.
`display.generate`, `display.press`	the two functions that would synthesise an action: both `ABSENT` on this model.
the `EVENT_*` bitmask	`PRESS`, `SCALE`, `DRAG`, `CURSOR`, `ENDAPP`, `SHOW`, `HIDE` are all **object** events. None is a key.

`ki.display`'s 29 action codes are a constant table with no Lua-visible consumer: the only function that took them, `display.generate`, is not registered here. Outside `ki.*` the whole 10.7 MB image contains exactly five key-related names — the four `display.KEY_*` constants and `display.EVENT_PRESS`.

So what the HELP key can do from a script is exactly one thing: `display.setkeyenable(display.KEY_HELP, display.OFF)` stops it opening the firmware's help screen. Useful for an app that wants the glass to itself; useless as an input.

The way to get a help screen is a button, not the key. The firmware already has the widget: the system information screen puts up a modal **scrollable text window with its own scrollbar and an OK button**, and `display.prompt(enum disp_buttons, string) -> int` is the entry point that builds one. Whether it scrolls arbitrary length text, and what the length limit is, is the next thing to measure. An app's own on-screen Help button calling `display.prompt` would give the same result the key would have, and it works today.

Every display signature, because the signatures are data

The argument lists do not have to be recovered one handler at a time. **Every name in the `display` namespace resolves to the same C closure** — one function with four upvalues, where upvalue 3 points at a 12-byte descriptor holding `{method_index, in_spec, out_spec}`, and each spec carries the minimum argument count, the maximum, and an array of per-argument type words. So a signature is a table lookup, and all 60 came out mechanically.

`min..max` is enforced **before** the handler runs, and a violation posts SCPI `-109` for too few arguments or `-108` for too many. On this instrument that is a pop-up, so an argument count is worth getting right from the table rather than by experiment.

There is no per-name C handler anywhere in the firmware, which is why this is the only route. All 387 function, getter and setter registrations push the *same* `lua_CFunction`, with one exception. The trampoline reads its upvalues and calls a dispatcher that is not a switch: it **marshals** the call, writing the command id and module id as 16-bit fields into a buffer for a service task. A TSP call is a message, not a function call. That explains the measured cost — about 4 µs before anything happens — and it is the same architecture as the display: work is handed to another executor.

Three things establish that the reading is correct rather than plausible. `display.input.prompt` comes out as `(enum disp_buttons, string) -> enum` and `display.input.number` as `(string, int, double, double, double, [string])`, both exactly the documented signatures. The switch cases in the implementation match the recovered method indices 0 to 58 one for one. And event `1130` is posted from this path as `(name, min, max)`, which is the same `1130` this project already measured when `display.create` refused a `digits` value outside 4..9.

In the table below, **argument counts and types are `CODE`** — read out of the firmware's own metadata. The argument *names* are a reading of the implementation and are weaker. The **exists** column is `CONFIRMED` or `ABSENT`, measured by `type()` on 1.7.17a. `obj` is a handle from `display.create`.

Lua name	arguments (min..max)	returns	exists
`display.type`	—	enum `disp_type`	property, reads `TYPE_BOTH`
`display.flush`	(obj) 1..1	—	function
`display.setcell`	(obj, int, int, string) 4..4	—	function
`display.setfont`	(obj, enum `disp_font`) 2..2	—	function
`display.setfill`	(obj, int percent, enum `disp_fill`) 2..3	—	function
`display.setminmax`	(obj, double, double) 3..3	—	function
`display.setpixel`	(int, int, int, value, int) **4..5**	—	function
`display.setthickness`	(obj, int thickness) 2..2	—	function
`display.setposition`	(obj, int, int, int, int) 3..5	—	function
`display.setabbrv`	(obj, string × 10) 3..11	—	function
`display.setevent`	(obj, enum `disp_event`, value) 3..3	—	function
`display.setpopupmenu`	(obj, int, enum state, string) 3..4	—	function
`display.setkeyenable`	(enum `disp_key`, enum `disp_onoff`) 2..2	—	function
`display.readingformat`	(enum `disp_readingformat`) 1..1	mirror	property, `FORMAT_PREFIX`
`display.lightstate`	(enum `disp_light_state`) 1..1	mirror	property, `STATE_LCD_25`
`display.watchchannels`	(string) 1..1	mirror	property, empty
`display.activebuffer`	(buffer) 1..1	mirror	table
`display.format`	(double, string, int, int, enum `disp_rbelement`) 4..5	string	not probed
`display.graph.add`	(obj, buffer, enum elem, value, value, enum elem, string) 1..7	—	function
`display.graph.scalex`	(obj, enum `disp_xscalemethod`, enum axis, double, double) 1..5	(double, double)	function
`display.graph.scaley`	(obj, enum `disp_yscalemethod`, enum axis, double, double) 1..5	(double, double)	function
`display.graph.scaletrace`	(obj, enum `disp_scaletrace`) 2..2	—	function
`display.graph.drawstyle`	(obj, enum `disp_drawstyle`) 2..2	—	function
`display.graph.cursor`	(obj, enum `disp_cursor`, double, double) 2..4	(double, double)	function
`display.graph.markersize`	(obj, int, int) 2..3	—	function

Lua name	arguments (min..max)	returns	exists
`display.graph.markerlastcolor`	(obj, enum, int) 2..3	—	function
`display.graph.select`	(obj, buffer, enum elem, string) 2..4	—	function
`display.graph.removeall`	(obj) 1..1	—	function
`display.generate`	(enum `disp_action`, int, int, int, int) 1..5	—	**absent**
`display.press`	(string, int, int) 1..3	—	**absent**
`display.getpixel`	(int x, int y) 2..2	int	**absent**
`display.getchecksum`	(int x1, int y1, int x2, int y2) 4..4	int	**absent**
`display.dump`	(enum `disp_dump`) 0..1	string	**absent**
`display.capture`	—	—	**absent**
`display.getobject`	(string) 1..1	object	**absent**
`display.autolocal`	(int) 1..1	mirror	**absent**

The graph family is registered **under** **`display.graph`**, not on `display` — which is why `display.scalex` reads nil while `display.graph.scalex` is a function. Two descriptors, method indices 38 and 52, carry full signatures and are registered under no name at all, so they are unreachable from Lua.

Enforced ranges recovered alongside the counts, each refused with `1130` naming its bound: `setthickness` is **1..10** inclusive, and `setfill`'s percent is **0..100**.

Three of the absent functions are worth knowing about anyway, because they would be the right tools on a model that has them. `getchecksum(x1,y1,x2,y2)` hashes a screen rectangle in **one** message, whereas `getpixel` blocks on a reply per pixel — so on an instrument with both, checksum a region and never poll pixels. `dump()` walks the object tree and returns a string. And `generate` is the one function in the module that is **not** read-only: it injects front-panel actions, where `ACTION_PRESS_FRONTREAR` switches the input terminals and `ACTION_PRESS_TRIGGER` starts the trigger model and can abort a running acquisition. It is absent here, which is the only reason that is a footnote rather than a warning.

No path in this module writes flash, EEPROM or NVRAM. Calibration is untouched by the display API.

Signatures worth noting against the documented ones: `display.create` is `2..16` arguments, `display.input.option` takes up to fourteen strings, `display.waitevent` is `0..1` and returns two or three values, and `display.clear` takes an **optional** object — `display.clear()` with no argument is legal.

Drawing fast: the graph object, measured against the pixel path

`CONFIRMED` on 1.7.17a. The question was whether an app can draw faster than the built-in graph by writing pixels. It cannot, and the graph is available to an app anyway.

`display.OBJ_GRAPH` is 18, and an app can create one on its own screen:

```
local scr = display.create(display.ROOT, display.OBJ_SCREEN, 'mygraph')
local g   = display.create(scr, display.OBJ_GRAPH, 0, 0, 780, 390)   -- exactly 6
        parameters
local b   = buffer.make(200, buffer.STYLE_WRITABLE)
buffer.write.format(b, buffer.UNIT_VOLT, buffer.DIGITS_6_5)
for i = 1, 200 do buffer.write.reading(b, math.sin(i / 12) * 5) end
display.graph.add(g, b)
display.changescreen(scr)
```

Measured, with the event log clean throughout:

step	time	note
200 `buffer.write.reading`	0.028 s	140 µs each
`display.graph.add(g, b)`	**0.004 s**	the firmware walks the buffer and draws
200 points, end to end	**~32 ms**	about 30 Hz
1,600 `setpixel` writes	0.020 s	12.7 µs each
`display.flush`	**0.203 s**	fixed -- the same for a 400x200 and an 800x480 object

So the pixel path is capped at about 5 Hz by a flush whose cost does not scale with the area drawn, while the graph path redraws 200 points at 30 Hz and brings axes, auto-scaling, grid, labels, line/marker/histogram styles and legends with it. Nothing an app writes per pixel competes with the firmware's own trace renderer, which runs in the UI task rather than in Lua.

Three things the create call will tell you if you get them wrong, each in its own event: `1714` "Object type Graph must have 6 parameters", `1130` "Parameter width, expected value from **120 to 799**" (so 800 is one too wide), and for `graph.add`, `1127` whose text is "reading buffer or channel list" -- the argument must be a reading buffer or a channel-list string.

A writable buffer is how an app plots arbitrary numbers. `buffer.make(n, buffer.STYLE_WRITABLE)` then `buffer.write.format(b, unit, digits)` then `buffer.write.reading(b, value)`. A standard-style buffer refuses the writes with event `4926`, "must be of a 'writable' style to use this function". The x axis is the buffer's **timestamps**, which is worth knowing before trying to make it a frequency axis: points written faster than the timestamp resolution bunch together.

display.setpixel writes the framebuffer directly

`CODE`. It is the one function in the module that does **not** post a message to the display task. The whole call ends in a single store:

```
((u32 *)(surface + 0x24))[x + stride * y] = (alpha << 24) | (colour & 0xFFFFFF);
```

```
display.setpixel(id, x, y, colour)          -- alpha defaults to 255
display.setpixel(id, x, y, colour, alpha)   -- alpha is the LOW BYTE of argument 5
```

So the pixel format is `0xAARRGGBB`, and **the top byte of argument 4 is discarded** — packing alpha into the colour does nothing; it has to be argument 5. The first argument is an object handle, like every other `set*`, so `setpixel(x, y, colour)` is refused with `-109`.

Since there is no message and no round trip, the cost of drawing this way is the Lua-to-C call itself — about 4 µs measured. That is a few milliseconds for a trace and over a second for a screenful, which is consistent with this project's own finding that `setpixel` was too slow to draw a spectrum with while the graph widget was not.

The coordinate range is per object, not 800x480. It comes from the object's own width and height, and an out-of-range coordinate posts `1130` quoting the real bound — so a deliberate out-of-range call is a way to ask an object how big it is. The object must also advertise a pixel capability or the call does nothing; `OBJ_IMAGE` is the likely candidate and which types qualify is unread.

setpixel has a bulk form that resamples an array over a whole object

`CODE` for the path and the resampling, `partial` for the Lua shape of the argument — and it is the most promising thing in this document. If argument 4 is an **aggregate** rather than a number, `setpixel` takes a different path: arguments 2 and 3 become a source **width and height**, argument 4 is read as a `srcW x srcH` array of doubles each holding a packed `0xRRGGBB`, and the firmware **bilinearly resamples it over the object's entire area**. Indices are clamped, so a source smaller than 2x2 is not useful.

One call would then paint an entire object with no per-pixel Lua overhead, which on a marshalled API where every other call costs microseconds is the difference between a drawable spectrum and a slideshow. What is not yet established is the exact Lua value argument 4 must be: the handler dereferences two levels before treating it as an array, which is consistent with a flattened table, but the flattener has not been read.

An out-of-range object handle may corrupt instead of failing

`HAZARD`, `partial` — stated carefully because the consequence is severe and the evidence is an inference rather than a read. Object handles are **10 bits**: `getpixel` clears those bits and the create path sets `id & 0x3FF`, both `CODE`. What follows from the field width, and has **not** been confirmed by reading a bounds check, is that a handle of 1024 would address object 0 rather than being refused — a stale or computed handle silently writing to a live object, with no event and nothing in the log. This project's pool is 463 objects, well inside the mask, so every handle it could hold aliases something real. Worth designing against; not yet worth asserting.

bit: the undocumented bitwise table, and when it is worth calling

`CONFIRMED`. `bit` is a global table on this instrument and holds three functions. Lua 5.0.2 has no bitwise operators, so this is the only way to get one without writing a loop. Nothing in any Keithley document mentions it.

```
bit.bitand(a, b)   bit.bitor(a, b)   bit.bitxor(a, b)
```

Measured, with the event log read either side of every call and **zero events** filed:

call	result	what it establishes
`bit.bitand(240, 60)`	48	
`bit.bitor(240, 15)`	255	
`bit.bitxor(255, 15)`	240	
`bit.bitand(0xFFFFFFFF, 0x12345678)`	305419896	the full 32-bit range works

call	result	what it establishes
`bit.bitor(2147483648, 1)`	2147483649	results come back unsigned, not sign-wrapped
`bit.bitand(255, 60, 15)`	60	**two arguments only** — a third is silently ignored

There is no shift, rotate or complement: `lshift`, `rshift`, `arshift` and `bnot` are all nil, as are `band` / `bor` / `bxor` — the names carry the `bit` prefix even inside the table. Shifts have to be done with `* 2^n` and `math.floor`, and a complement with `bit.bitxor(x, 0xFFFFFFFF)`.

It is not automatically a speed win, and for masking it is a loss. 20,000 iterations each, loop overhead subtracted:

loop body	per iteration
`x = bit.bitand(i, 255)`	4.1 µs
`x = bit.bitxor(i, 255)`	4.1 µs
`x = math.mod(i, 256)`	3.5 µs
`x = i - math.floor(i / 256) * 256`	**2.7 µs**

A Lua-to-C call on this interpreter costs about 4 µs — roughly 1,600 cycles at 400 MHz — and that dominates the operation itself. So:

- **Masking to low bits: use arithmetic.** `i - math.floor(i/256)*256` is 1.5x faster than `bit.bitand(i, 255)`, and `math.mod` is faster too. The call overhead is the whole cost.
- **A general AND or OR of two arbitrary values: use `bit`.** The arithmetic equivalent needs a loop over bit positions, which is many times 4 µs.
- **XOR: use `bit.bitxor` always.** There is no cheap arithmetic equivalent at all.

The rest of that table is not reachable from Lua

The same internal table in the image carries `memread`, `memwrite`, `peek8`, `peek16`, `peek32`, `poke8`, `poke16`, `poke32`, `spindelay`, `trace`, `intercept`, `aborthook`, `remotecomm` and a flash protection call. **Every one of them is nil on this firmware**, under both `_G` and the `memory` table, which is itself a table with none of these members. `IMAGE` for their existence, `ABSENT` for their reachability.

That is worth stating precisely because of what they would be able to do. A poke or a flash protection call reached from a script is the one thing on this instrument that could destroy calibration data irrecoverably, and the measured answer is that Lua has no entry point to them. No argument list for any of them is recorded here, and none should be worked out: the names are listed so that nobody goes looking for them, not so that anybody can use them.

Namespaces nothing has mentioned

`IMAGE`, from the registration walk: 63 namespaces in all, against the 44 a dotted-string count suggests. Each of these holds registered constants rather than being a loose string: `serial` 24, `sysinfo` 23, `analogtrigger` 12, `interruptcontroller` 14, `interruptmapping` 7, `format` 9, `coupling` 3, `filter` 4, `rel` 3, `threshold` 2, `fan` 2, `offsetcompensation` 4, plus `mpc512x` — which names the SoC, a Freescale MPC512x with an e300 core. Six have no documentary trace of any kind: `emu`, `flog`, `c2c`, `backgrounds`, `tmodel`, `sysinfo`.

`ABSENT`, and this is the shared-codebase rule again: `sysinfo`, `emu`, `flog`, `c2c`, `tmodel` and `backgrounds` are all nil as globals on a DMM6500 at 1.7.17a. Registered in the image, not reachable here.

`interruptcontroller` and `interruptmapping` are the ones to look at next: if the mailbox between the application CPU and the graphics processor is reachable from Lua, it is reachable through those.

The event number table

`IMAGE`. **676 event codes with their message text and, for most, a detail paragraph** — `docs/firmware-events.tsv`. 565 errors, 67 warnings, 44 information. This is the single most useful thing recovered from the image for day-to-day work: every unexplained number this project has chased is in it.

12-byte records in `.data`, anchored at link `0x009B098C`:

```
struct { uint32_t codeword; const char *message; const char *detail; };
// codeword = (int16 number << 16) | (severity << 8) | flags
```

The number is the top 16 bits **read as a signed int16**, which is how the negative SCPI errors share one table with the positive Keithley events. Severity 1/2/4 matches the documented `eventlog` constants.

Validated independently of everything else here: twelve standard SCPI codes come out verbatim — `-100` Command error, `-104` Data type error, `-108` Parameter not allowed, `-109` Missing parameter, `-200` Execution error, `-222` Parameter data out of range, `-363` Input buffer overrun and others.

Every code this project has ever hit resolves, including the five provoked while writing this file:

code	message
`1127`	Parameter, %s, expected %s data type
`1130`	Parameter %s, expected value from %d to %d
`1138`	Parameter %d, expected type '%s', but found type '%s'
`1703`	Object id is invalid — *detail*: "never created or has been destroyed"
`1715`	Object type %s must have between %d and %d parameters
`1716`	Object type %s parameter %d must be type %s
`1717`	Attribute %s, does not apply to an object of type %s
`2201`	File read error
`2205`	File not found
`2351`	SPLT non-critical message lost
`2710`	Unable to initiate, no block number %d found in trigger model
`4915`	Attempting to store past the capacity of reading buffer
`4926`	Reading buffer %s must be of a 'writable' style to use this function

Two codes seen on this bench are **not** in the table and therefore come from a different mechanism: `-285` (TSP syntax error) and `5076`.

`1703`'s detail settles something in this project's own history: the stale-display-object-handle event says in as many words that the id "was never created or has been destroyed".

bit: ten functions, not three

`IMAGE` for the full set, `CONFIRMED` for the first three. Each has its own real C function rather than the shared trampoline, so the signatures come from each handler's own argument checks. They match Keithley's documented 2600-series bit library one for one, which is the independent check that the extraction is right — and no DMM6500 manual mentions `bit` at all.

function	signature	returns
`bit.bitand`, `bit.bitor`, `bit.bitxor`	(number, number)	number
`bit.test`	(value, index)	**boolean**
`bit.get`, `bit.set`, `bit.clear`, `bit.toggle`	(value, index)	number
`bit.getfield`	(value, index, width)	number
`bit.setfield`	(value, index, width, fieldValue)	number

Every argument is type-checked and raises `-104` "Data type error" on a non-number; the index and width family raises `-222` "Parameter data out of range". `bitand` / `bitor` / `bitxor` carry an explicit unsigned-32 fixup, so values at or above 2^31 are handled rather than truncated — which is what the measured `bit.bitor(2147483648, 1) = 2147483649` shows. There is no `bitnot`.

`bit.getfield` and `bit.setfield` are the interesting pair for this project: extracting a bit field from a decoded byte is one call instead of a divide, a floor and a modulo.

Why the memory family is unreachable, mechanically

`memread`, `memwrite`, `peek8/16/32`, `poke8/16/32`, `trace`, `intercept`, `aborthook`, `remotecomm`, the flash-protection call and `pseudocal` are all registered by **one** function whose registrar has **zero callers anywhere in `.text`**. Not merely absent from `_G` — unreachable by construction, because nothing ever invokes the code that would publish it. That is a stronger statement than the `nil` measurements and it explains them.

Two names that both the `cal` and `nvram` token rules miss, and which are **not** protected by being dead code: `sysinfo.nvmeminit` and `sysinfo.nvmemdump`, ordinary registrations in a live registrar. `nvmem` is now in the withheld set. No argument lists for either are recorded anywhere.

Flush is a bulk copy, and the image surface is owned by its name

`CODE` and `CONFIRMED`, and the two together resolve a contradiction in my own measurements.

`display.flush(id)` is not a small message. It copies the image object's **entire pixel block** to the display processor over channel 9 in **240-byte chunks** under a per-channel lock. So its cost is proportional to the surface's **area**: an 800x480 32-bit surface is 1,536,000 bytes, 6,400 chunks.

I measured 0.203 s for a 400x200 object and 0.203 s for an 800x480 one, which looks like a fixed cost and contradicts that — until the second finding explains it. **The string argument to `display.create(..., OBJ_IMAGE, x, y, name, w, h)` names a surface, and the surface is what owns the pixels.** A 400x200

object created with a name already bound to an 800x480 surface displays and flushes that 800x480 surface; the `w` / `h` arguments did not resize it. Both of my flushes were copying the same surface, so both cost the same. The static read stands.

A name that has no surface gets a placeholder. Creating `OBJ_IMAGE` with a fresh name renders a small blue "?" icon, and `setpixel` into it writes nothing and posts **no event at all** — measured with 60,000 calls. On a freshly booted instrument the first fresh name allocated a surface and worked; later fresh names got the placeholder, which suggests the surface is not released when the object is deleted. So an app should create one image at startup, keep it, and size it to the region it actually redraws.

The capability allow-list for `setpixel`, for its bulk form, and for `flush` each contain **exactly one** object type: 5, `OBJ_IMAGE`. Nothing else has a pixel surface, and `OBJ_RECT` posts `1717` per call rather than failing silently.

The stale handle hazard, corrected

`CODE`. The earlier claim that a handle of 1024 aliases object 0 is **retracted**: the capability check does bounds-check, and a handle at or above 512 is refused with `1130` quoting 0..511.

The real hazard is quieter and worse. A handle in 0..511 whose **slot has been reused** passes every check, because there is no generation counter and no ownership test — the record only has to be non-zero and of an accepted type. The call then edits a live, unrelated object and reports success. With 463 objects against 512 slots, reuse is the normal case, and for the narrow allow-lists above a stale handle corrupts some *other* image. Null a handle at the point of delete.

Still open

- Every signature. The names and constants above are solid; the argument lists are not, and are being recovered from the registrar's metadata and the handlers rather than guessed.
- `display.flush` — whether it batches drawing. This is the single most useful thing left.
- What consumes the `ki.display` action codes, which is the HELP key answer.
- `logic.zx` and `rom.zip`, neither opened. `logic.zx` is the best candidate for the raster engine.
- The event-number table. A code-to-text list for 2201, 2205, 2710, 1130, 1138, 1703, 1716 and 4915 would have saved this project hours.